UNIVERSITY OF WARWICK

Paper Details

Paper Code: CS2580

Paper Title: Database Systems

Exam Period: April 2026

Exam Rubric

Time Allowed: 2 hours

Exam Type: Standard Examination

Approved Calculators: Permitted

Instructions

Answer **ONLY FOUR** questions.

Read carefully the instructions on the answer booklet and make sure that the particulars required are entered on each answer booklet.

The exam question paper **MUST NOT** be removed from the examination venue.

Permitted calculators: CASIO FX82, FX83 and FX85 series.

Question 1 **Writing SQL Queries.** [25 marks]

A cinema chain stores data in the following relations:

- Cinema(cinema_id, name, city)
- Screen(screen_id, cinema_id, capacity)
- Film(film_id, title, certificate)
- Showing(showing_id, film_id, screen_id, show_date, start_time)
- Ticket(ticket_id, showing_id, customer_id, price_paid)
- Customer(customer_id, full_name, birth_year)

Note: capacity, birth_year, price_paid are numeric; show_date is a date; start_time is a time. certificate is the film's age rating/classification (e.g., U, PG, 12A, 15, 18). Each ticket corresponds to exactly one showing.

Unless explicitly stated, you must **not** assume any additional keys, uniqueness constraints, or `NOT NULL` constraints beyond those implied by the schema (i.e., attributes may be non-unique and/or `NULL`).

(a) [6]

Write a SQL query to compute, for each cinema in Warwick, the total revenue (sum of `price_paid`) from tickets sold for showings in April 2025 (2025-04-01 to 2025-04-30 inclusive).

Return: cinema name, total_revenue. Results should be in descending order by total_revenue.

Constraints: Must use explicit joins. No subqueries are allowed.

(b) We want to find films that have been shown in every cinema in London. To make this easier, define a helper view `Missing(film_id, cinema_id)` that contains exactly the pairs (f, ci) such that ci is a London cinema and film f has never been shown in ci.

i. [6]

Write a SQL query that returns `Missing(film_id, cinema_id)`.
Constraints: Must use explicit JOINs. Must also use `NOT EXISTS` (no `NOT IN`, no outer joins with `IS NULL`, no set operators).

ii. [4]

Using `Missing`, write a SQL query to list the films that have been shown in every cinema in London.
Return: `film_id, title`.
Constraint: Must use **only one** `NOT EXISTS` and no other subqueries (no `NOT IN`, no set operators).

(c) [9]

Write a SQL query to find pairs of different customers (A, B) who:

- attended the same showing (both bought tickets for the same `showing_id`), and
- have birth years within 2 years of each other, and
- return each unordered pair once (i.e., treat (A, B) and (B, A) as the same).

Return: `showing_id, A.full_name, A.birth_year, B.full_name, B.birth_year`.

Constraints: Must use explicit JOIN ... ON ...; No subqueries are allowed.

Question 2 **Understanding SQL Queries.** [25 marks]

Using the cinema schema from Q1, answer each question below in **1–2 sentences**.

(a) [6]

What does the following query return exactly?

```
SELECT c.customer_id
FROM Customer c
WHERE EXISTS (
  SELECT *
  FROM Ticket t
  JOIN Showing s ON s.showing_id = t.showing_id
  WHERE t.customer_id = c.customer_id
  GROUP BY t.customer_id
  HAVING COUNT(DISTINCT s.film_id) >= 3
);
```

(b) [7]

Consider Query A and Query B below.

```
-- Query A
SELECT c.customer_id
FROM Customer c
WHERE c.customer_id NOT IN (
  SELECT t.customer_id
  FROM Ticket t
);

-- Query B
SELECT c.customer_id
FROM Customer c
WHERE NOT EXISTS (
  SELECT *
  FROM Ticket t
  WHERE t.customer_id = c.customer_id
);
```

Do Query A and Query B always return the same result? If you believe **Yes**, explain why in 1–2 sentences. If you believe **No**, give a **concrete counterexample** by specifying **one tuple** in `Customer(customer_id)` and **one or two tuples** in `Ticket(customer_id)` for which the two queries return different results (it is enough to identify one `customer_id` that differs).

(c) Consider Query A and Query B below.

```
-- Query A
SELECT ci.name, COUNT(t.ticket_id) AS tickets
FROM Cinema ci
LEFT JOIN Screen sc  ON sc.cinema_id = ci.cinema_id
LEFT JOIN Showing s  ON s.screen_id = sc.screen_id
LEFT JOIN Ticket t   ON t.showing_id = s.showing_id
WHERE ci.city = 'Manchester'
  AND s.show_date = '2025-04-01'
GROUP BY ci.name;

-- Query B
SELECT ci.name, COUNT(t.ticket_id) AS tickets
FROM Cinema ci
LEFT JOIN Screen sc  ON sc.cinema_id = ci.cinema_id
LEFT JOIN Showing s  ON s.screen_id = sc.screen_id
                    AND s.show_date = '2025-04-01'
LEFT JOIN Ticket t   ON t.showing_id = s.showing_id
WHERE ci.city = 'Manchester'
GROUP BY ci.name;
```

i. [6]

In **1–2 sentences each**, explain what Query A returns and what Query B returns.

ii. [6]

Do Query A and Query B always return the same result? Answer Yes/No.
If you answered **Yes**, justify briefly (1–2 sentences).
If you answered **No**, give a **concrete counterexample** (a tiny instance with 1–2 tuples per relevant table is enough) for which the outputs differ, and state what each query returns on your instance.

Question 3 **DB Security.** [25 marks]

A DB uses the following randomised response scheme for a sensitive YES/NO attribute:

- Flip a biased coin with probability $p_H = P(\text{HEADS})$.
- If the coin shows HEADS, the DB stores the true answer (YES or NO).
- If the coin shows TAILS, the DB stores YES, regardless of the truth.

Let θ be the true fraction of YES in the population, and $\hat{\theta}$ be the final estimate of θ computed from the stored (noisy) answers. We define:

- **Privacy** = how unlikely it is that the stored answer equals the true answer (higher mismatch $\Rightarrow$ more private).
- **Accuracy** = how close our final estimate $\hat{\theta}$ is to the estimate we would get without randomisation.

Consider three possible values: $p_H \in \{0.2, 0.5, 0.8\}$.

(a) [3]

From a privacy perspective, for which value(s) of p_H is the scheme more private? Briefly justify your answer in 1–2 sentences.

(b) [3]

Using the definition of accuracy above, which value(s) of p_H make the scheme more accurate? Again justify briefly in 1–2 sentences.

(c) [3]

A company database stores salaries: Employees(eid, name, dept, gender, level, salary). The 4 employees in department CS are:

eid	name	dept	gender	level
1	Alice	CS	F	Senior
2	Bob	CS	M	Senior
3	Carol	CS	F	Junior
4	Dave	CS	M	Junior

The DBMS enforces a privacy rule: only aggregation queries (COUNT, SUM, AVG) on salary are allowed, and every query's WHERE clause must match at least 3 CS employees (otherwise the DBMS returns `suppressed`).

You are an attacker who knows all non-salary attributes above, but not the salaries. You want to infer Alice's salary.

Consider the following candidate query sets (each query is of the form:

```
SELECT SUM(salary) FROM Employees
WHERE dept='CS' AND name IN (...); ):
```

- Set A
  - Q1: name IN ('Alice','Bob','Carol')
  - Q2: name IN ('Alice','Bob','Dave')
- Set B
  - Q1: name IN ('Alice','Bob','Carol')
  - Q2: name IN ('Bob','Carol','Dave')
- Set C
  - Q1: name IN ('Alice','Bob','Carol')
  - Q2: name IN ('Alice','Bob','Dave')
  - Q3: name IN ('Alice','Carol','Dave')
  - Q4: name IN ('Bob','Carol','Dave')

Each query in any set is allowed by the DBMS (they all match 3 CS employees). Which set (A, B, or C) allows you to uniquely determine Alice's salary in principle? Briefly justify your choice in 1–2 sentences.

(d) [7]

Assume you use the set you chose in part (c), and the DBMS returns the following results:

- S_{ABC} = SUM(salary) for name IN ('Alice','Bob','Carol') = 210000
- S_{ABD} = SUM(salary) for name IN ('Alice','Bob','Dave') = 210000
- S_{ACD} = SUM(salary) for name IN ('Alice','Carol','Dave') = 200000
- S_{BCD} = SUM(salary) for name IN ('Bob','Carol','Dave') = 190000

Let Alice, Bob, Carol, Dave have salaries a, b, c, d respectively.

(i) Give a formula for a in terms of $S_{ABC}, S_{ABD}, S_{ACD}, S_{BCD}$.

(ii) Using the numbers above, compute a.

(e) [4]

Consider the following JDBC code. Assume the DBMS/driver allows multiple SQL statements in a single query string.

```
String baseSql =
    "SELECT id, name, salary " +
    "FROM Employees " +
    "WHERE dept = ? ";

if (!sortField.isEmpty()) {
    baseSql = baseSql + "ORDER BY " + sortField + " ";
}

PreparedStatement ps = conn.prepareStatement(baseSql);
ps.setString(1, dept);

ResultSet rs = ps.executeQuery();
```

Is this code safe from SQL injection? Briefly justify your answer in 1–2 sentences.

(f) [5]

If your answer to part (e) above is No, give a malicious value for `sortField` and write the resulting SQL string when `dept = 'CS'`. If your answer is Yes, explain in one sentence why.

Question 4 **Functional Dependencies.** [25 marks]

Let $R(A, B, C, D, E, F)$ with functional dependencies $\mathcal{F}$:

$$A \to BC, \quad C \to A, \quad BD \to E, \quad E \to F.$$

(a) [6]

(i) Does $A \to B$ imply a 1–1 relationship between A and B? Answer Yes/No and give a 1-sentence explanation.

(ii) Using the given FDs, is the mapping between A and C 1–1? Answer Yes/No. Justify by citing the relevant dependencies.

(b) [7]

For each statement below, answer True/False (implied by $\mathcal{F}$ or not). Explain with 1 sentence each answer.

(i) $A \to E$

(ii) $D \to E$

(iii) $BD \to F$

(iv) $C \to BC$

(c) [7]

Compute $(BD)^+$ and $(CD)^+$ under $\mathcal{F}$. Then list all candidate keys of R.

(d) [5]

Is R in BCNF under $\mathcal{F}$? If you answer No, decompose R into BCNF using a violating FD you identify (give the resulting relation schemas only). Then answer:

(i) Is your decomposition lossless-join? Answer with Yes or No. (1 sentence justification.)

(ii) Is your decomposition dependency-preserving for the original FD set $\mathcal{F}$? Answer with Yes or No. (1 sentence justification.)

Question 5 **Relational Algebra and Relational Calculus.** [25 marks]

Consider the following schema:

- Student(sid, name, major)
- Course(cid, title, dept)
- Enrol(sid, cid, term, grade)
- Prereq(cid, prereq_cid) (direct prerequisites only)

Assume `sid` and `cid` are keys for `Student` and `Course` respectively.

(a) [6]

Write a relational algebra expression to return the names of students who are enrolled in **at least one** course offered by the **CS** department in term **'2025S'**, but whose major is **not** CS.

Return: $\{ name \}$.

(b) [7]

Write a relational algebra expression to find students (sid, name) such that:

For **every** CS course c that the student has enrolled in (in any term), and for **every** direct prerequisite p of c, the student has also enrolled in p (in any term).

Return: $\{ sid, name \}$.

(c) [6]

Write a Tuple Relational Calculus (TRC) query to return the set of titles of CS courses that are **not a prerequisite of any course** (i.e., their `cid` never appears as `prereq_cid` in `Prereq`).

Return: $\{ title \}$.

(d) [6]

Write a Tuple Relational Calculus (TRC) query to return the names of students who, in term **'2025S'**, received grade **'A'** in **every CS course** they took that term.

Return: $\{ name \}$.

End